

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

DOP203-R1

Applying Amazon's DevOps culture to your team

Leo Zhadanovsky

Chief Technologist, Education
AWS

Steve Rice

General Manager, AWS AppConfig
AWS



Introductions



Leo Zhadanovsky
Chief Technologist, Education
AWS



Steve Rice
General Manager, AWS AppConfig
AWS

What we will learn today

- How Amazon builds organizations, processes, and systems that enable rapid innovation and delivery of high-performing digital solutions for its customers
- Lessons learned at Amazon for developing and delivering software
- Amazon's best practices for building, testing, deploying, and maintaining modern applications
- Q&A throughout

Lessons learned at Amazon for developing and delivering software



Decompose for agility
(**microservices, two-pizza teams**)



Automate everything



Standardized tools



Belts and suspenders
(**governance, templates**)



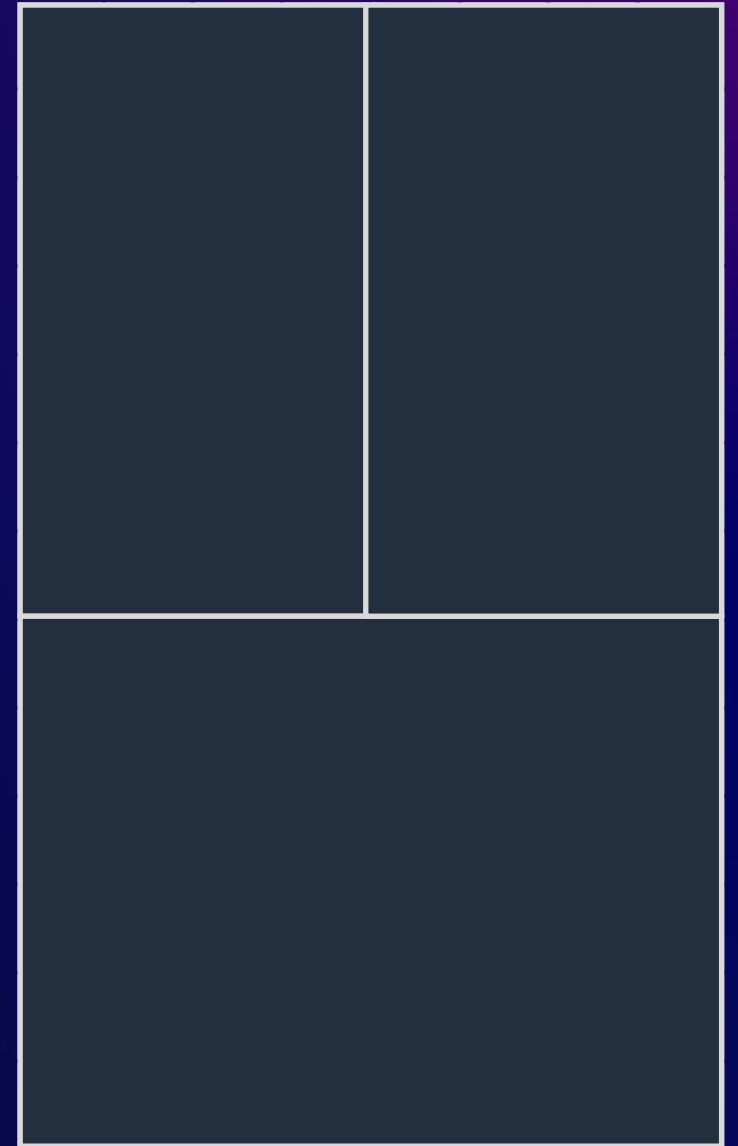
Infrastructure as code



Continuous configuration
(**feature flags, operational flags**)

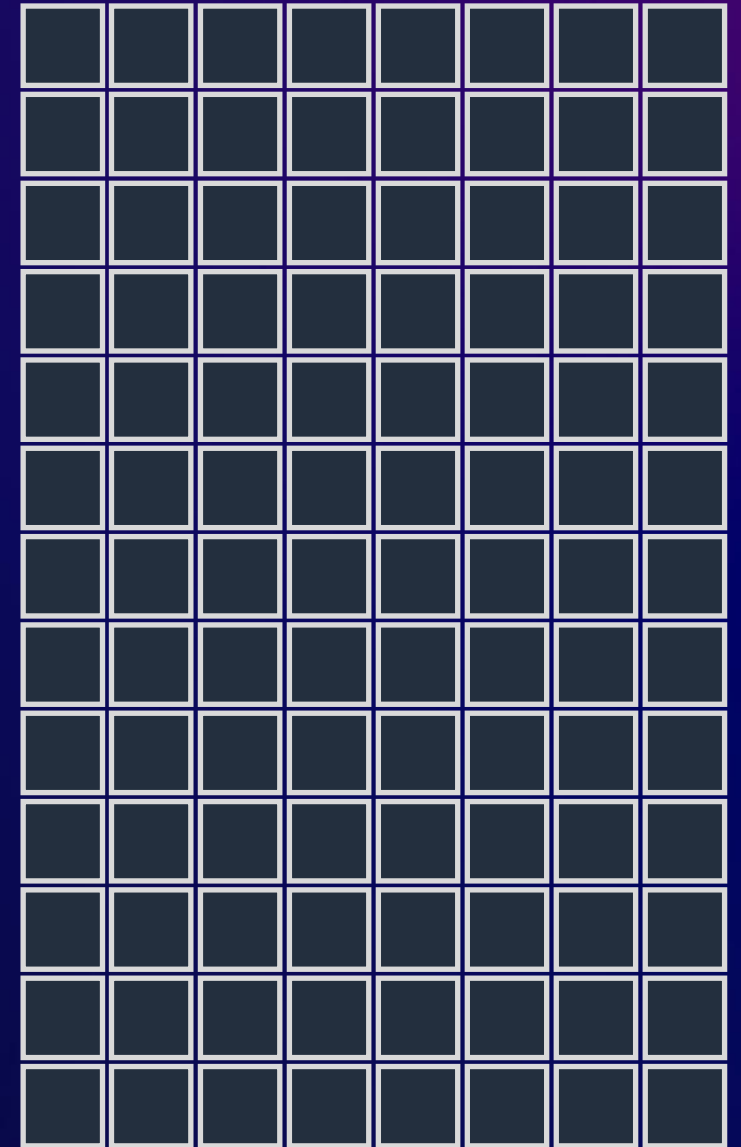
Decompose for agility

- Principles
- Make units as small as possible (primitives)
- Decouple based on scaling factors, not functions
- Each service operates independently
“Communication is terrible!” – Jeff Bezos
- APIs (contracts) between services



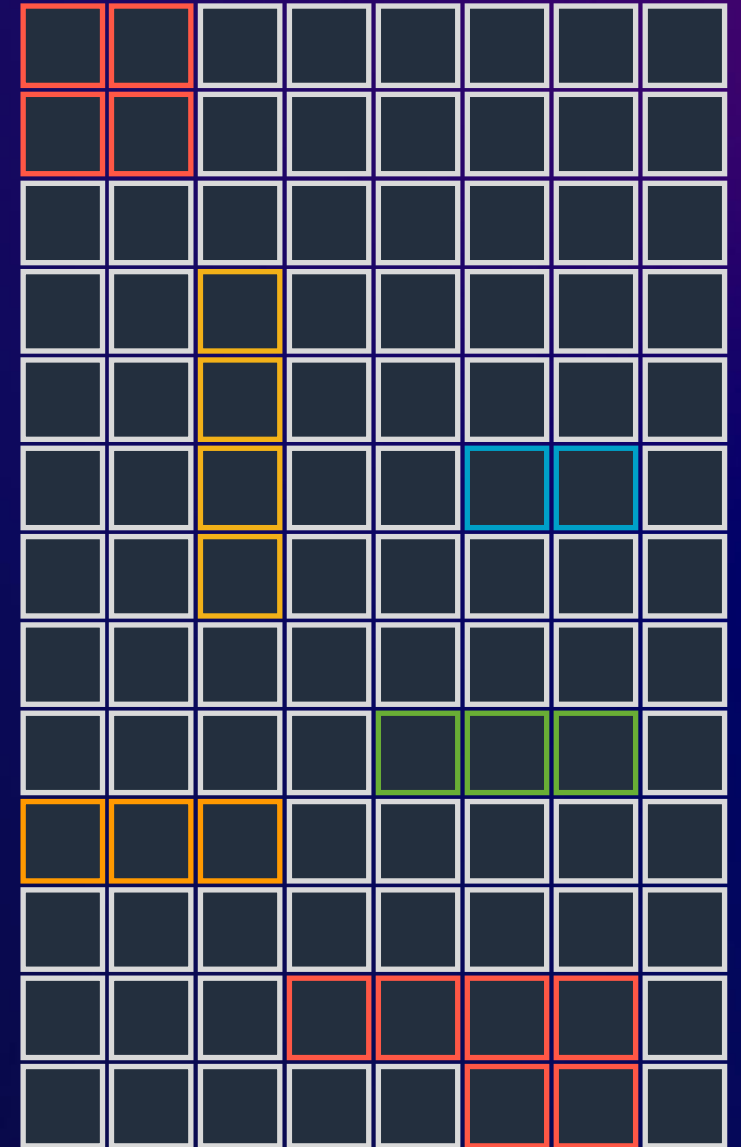
Decompose for agility

- Principles
- Make units as small as possible (primitives)
- Decouple based on scaling factors, not functions
- Each service operates independently
"Communication is terrible!" – Jeff Bezos
- APIs (contracts) between services



Decompose for agility

- Principles
- Make units as small as possible (primitives)
- Decouple based on scaling factors, not functions
- Each service operates independently
“Communication is terrible!” – Jeff Bezos
- APIs (contracts) between services



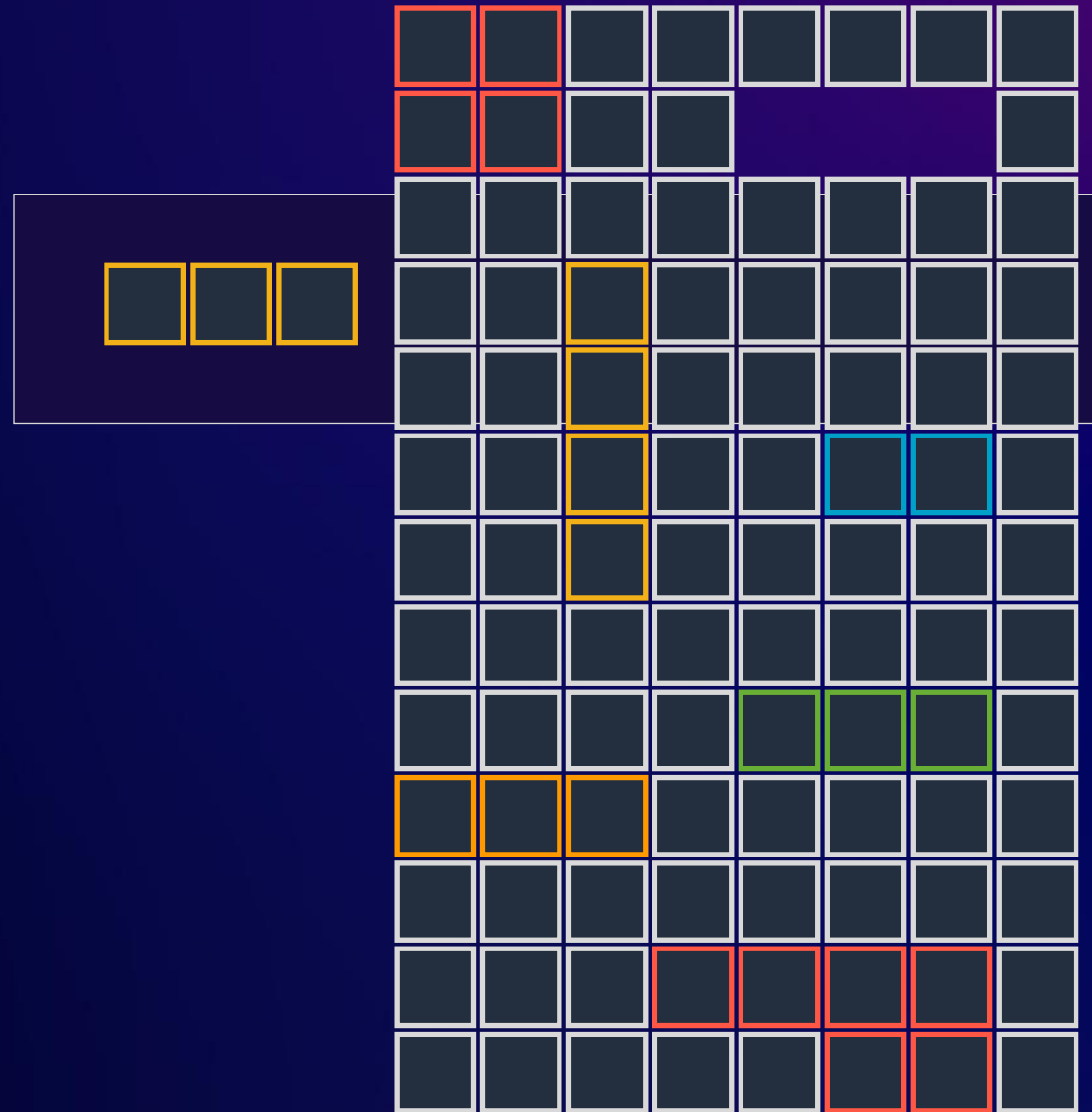
Decompose for agility

- Principles
- Make units as small as possible (primitives)
- Decouple based on scaling factors, not functions
- Each service operates independently
"Communication is terrible!" – Jeff Bezos
- APIs (contracts) between services

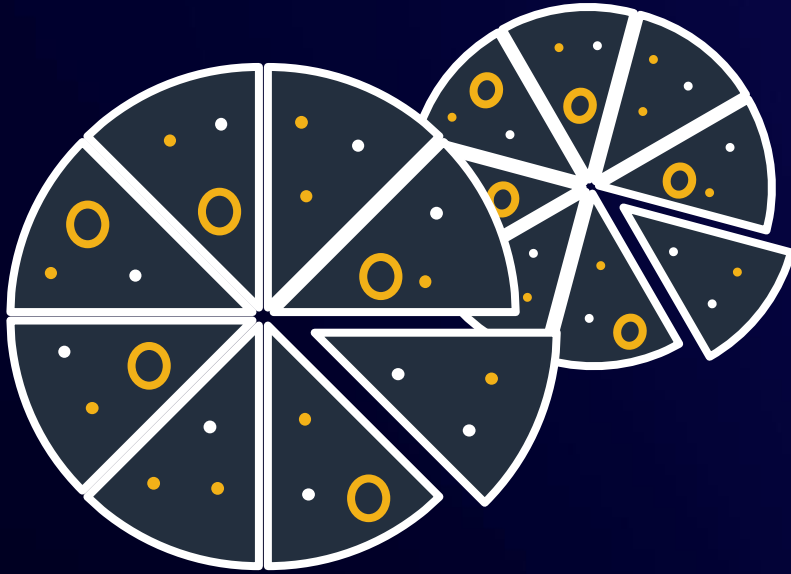


Decompose for agility

- Principles
- Make units as small as possible (primitives)
- Decouple based on scaling factors, not functions
- Each service operates independently
“Communication is terrible!” – Jeff Bezos
- APIs (contracts) between services



Getting (re)organized



“Two-pizza” teams

- Own a service
- Minimize social constraints (Conway’s law)
- Autonomy to make decisions
- Manage communication
comms channels = $n(n-1)/2$



2 people



5 people



12 people

Teams own everything

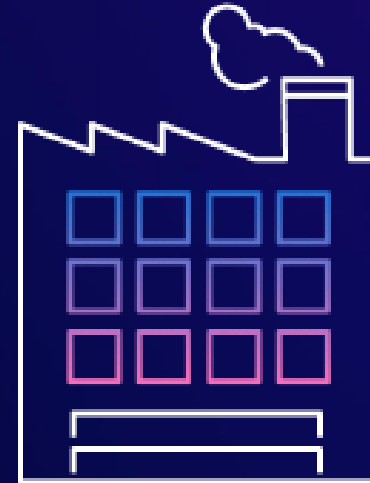
- Planning
- Security
- Performance
- Scalability
- Deployment
- Operation
- Bugs
- Documentation
- Testing



How can one team do all of this?

Automate everything

- CI/CD
- Build
- Testing
- Deployments
- Governance
- Feature flags
- DevOps
- Patching
- IaC



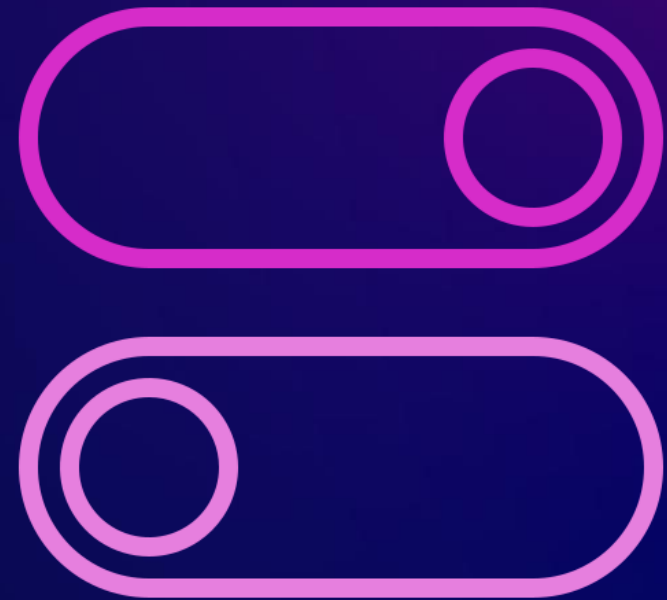
Automation = developer force multiplier

Learn how AWS uses automation for patching:



Continuous configuration

- Goes beyond what CI/CD can do
- Teams move faster
- Deployments are safer
- Separate a release from a code deployment
- Feature flags
- Operational flags



Learn how AWS uses
continuous configuration:



Measuring and enabling success

Business metrics	Operational metrics	Input goals	Enablement
Growth	Errors	Features	Principal reviews
Usage	Throttling	Use cases	Security training
Feedback	Failed deployments	Performance	Ops training
	Performance		Platform tooling

Well-architected mechanisms

1. **Correction of errors (COE)** – Blameless autopsy after an operational event
2. **Operational readiness review (ORR)** – Checklist for “what ifs” done annually and before a major release
3. **Weekly Ops meeting with Spin the Wheel** – Randomization increases engagement



Learn how to conduct an ORR for your workloads



Modern applications

Today we have modern applications that



- Use independently scalable microservices (serverless, containers)
- Connect through APIs
- Deliver updates continuously
- Deployed through IaC
- Adapt quickly to change
- Scale globally
- Are fault tolerant
- Carefully manage state and persistence
- Have built-in security
- Can be tuned on production without deploying code
- Use learnings from internal services to build and improve external services

Q&A



I want my organization to build modern applications – Where do I get started?

**I want to move from a monolith
to a microservices architecture –
Where do I begin?**

How do I maintain security and compliance in my production CI/CD pipelines while giving my developers the ability to innovate and be agile in their development environments?

How do feature flags allow me to move faster and safer? Don't they just clutter up my code?

Can I still have separate Application and Ops teams and still take advantage of these?

How do I restructure my teams for building modern applications?

Thank you!

Leo Zhadanovsky

leozh@amazon.com

 leozh

Steve Rice

sjrice@amazon.com

linkedin.com/in/stevejrice/



Please complete the session survey in the mobile app